

Difference between JPA, Hibernate and Spring Data JPA

Basic Definition:

- **JPA (The Rulebook):** A specification (set of guidelines) on how to handle persistence in Java. It defines the interfaces but does nothing on its own.
- **Hibernate (The Implementation):** A tool that actually does the work. It reads the JPA "rulebook" and provides the code to move data between Java objects and the database.
- **Spring Data JPA (The Enhancer):** A layer on top of Hibernate that automates repetitive tasks (like writing SQL queries or managing sessions) so you can focus on business logic.

Difference Table:

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification (API)	ORM Framework	Data Access Abstraction
Analogy	A blueprint for a car.	The actual physical car.	An autopilot system for the car.
Boilerplate	N/A (it's just interfaces)	High (requires Session/Transaction mgmt)	Very Low (handled via Repositories)
Dependency	None	Implements JPA	Requires a JPA provider (like Hibernate)

Differentiation through Code:

User.java(Common Code in all the three):

```
import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
```

```

public User() {}
public User(String name) { this.name = name; }
public Long getId() { return id; }
public String getName() { return name; }
}

```

1. JPA:

persistence.xml:

```

<persistence-unit name="my-jpa-unit">
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <class>com.example.User</class>
  <properties>
    <property name="jakarta.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/mydb"/>
    <property name="jakarta.persistence.jdbc.user" value="root"/>
  </properties>
</persistence-unit>

```

JpaExample.java:

```

import jakarta.persistence.*;

public class JpaExample {
  public static void main(String[] args) {
    EntityManagerFactory emf = Persistence.createEntityManagerFactory("my-jpa-unit");
    EntityManager em = emf.createEntityManager();
    em.getTransaction().begin();

    User user = new User("John Doe");
    em.persist(user); // Standard JPA method

    em.getTransaction().commit();

    em.close();
    emf.close();
  }
}

```

2. Hibernate:

hibernate.cfg.xml:

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/mydb</property>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <mapping class="com.example.User"/>
  </session-factory>
</hibernate-configuration>
```

HibernateExample.java:

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateExample {
  public static void main(String[] args) {
    SessionFactory factory = new Configuration().configure().buildSessionFactory();

    Session session = factory.openSession();

    try {
      session.beginTransaction();

      User user = new User("Jane Doe");
      session.save(user);

      session.getTransaction().commit();
    } finally {
      session.close();
      factory.close();
    }
  }
}
```

Spring Data JPA:

UserRepository.java:

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

// No implementation code needed! Spring creates it automatically.
@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    User findByName(String name);
}
```

UserService.java:

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Transactional // Automatically handles begin, commit, and rollback
    public void createUser() {
        User user = new User("Spring User");
        userRepository.save(user);
    }
}
```